```
    Behaviors.receive { (context, message) =>
      message match {
        case Increase =>                                          ❶
          context.log.info(s"wallet is deactivated. Can't increase")
          Behaviors.same
        case Deactivate =>                                        ❷
          Behaviors.same
        case Activate =>                                          ❸
          context.log.info(s"activating")
          activated(total)
      }
    }
  }
}
```

❶ Logs that it can’t increase

❷ Keeps the same behavior with the same total

❸ Changes to the activated behavior with the same total

The code for the entire actor now looks like the following listing.

Listing 2.8 `WalletOnOff` actor

```
object WalletOnOff {

  sealed trait Command
  final case class Increase(amount: Int) extends Command
  final case object Deactivate extends Command
  final case object Activate extends Command

  def apply(): Behavior[Command] = activated(0)                   ❶

  def activated(total: Int): Behavior[Command] =                  ❷
    Behaviors.receive { (context, message) =>
      message match {
        case Increase(amount) =>
          val current = total + amount
          context.log.info(s"increasing to $current")
          resume(current)
        case Deactivate =>
          deactivated(total)
        case Activate =>
          Behaviors.same
      }
    }
```